

Red Social Web

Proyecto Final de Grado · DAM · Fabio Cuffaro Cámara

Una red social full-stack construida desde cero

- Registro
- Autenticación JWT
- Posts con imágenes
- Feed infinito
- Perfiles con sistema de follows
- Búsqueda de perfiles
- Me gusta en tiempo real
- Asincronia en Peticiones...

¡Bienvenido de nuevo!
Inicia sesión para continuar

Email

Contraseña

Iniciar Sesión

¿No tienes cuenta? [Regístrate aquí](#)

Motivación y Tecnologías

¿Por qué este proyecto y no otro?

El problema: Aprender Backend real es complicado.

Objetivos: 1º Auth JWT de principio a fin. 2º Feed paginado sin el problema N+1. 3º Imágenes con credenciales seguras. 4º Tests contra PostgreSQL real vs. alternativas existentes: Firebase o Auth0 ocultan lo que quería aprender.

Lo he implementado todo desde cero.

Funcionalidades implementadas

- Registro e inicio de sesión con JWT
- Posts con imagen subida a Cloudinary
- Feed global paginado con scroll infinito
- Sistema de follows entre usuarios
- Likes con actualización en tiempo real
- Búsqueda de usuarios en tiempo real
- Tests de integración automatizados con PostgreSQL real

Backend

Java 17 + Spring Boot 3, Spring Security + JWT, JPA/Hibernate, Lombok, Bean Validation

Base de Datos

PostgreSQL como motor relacional principal

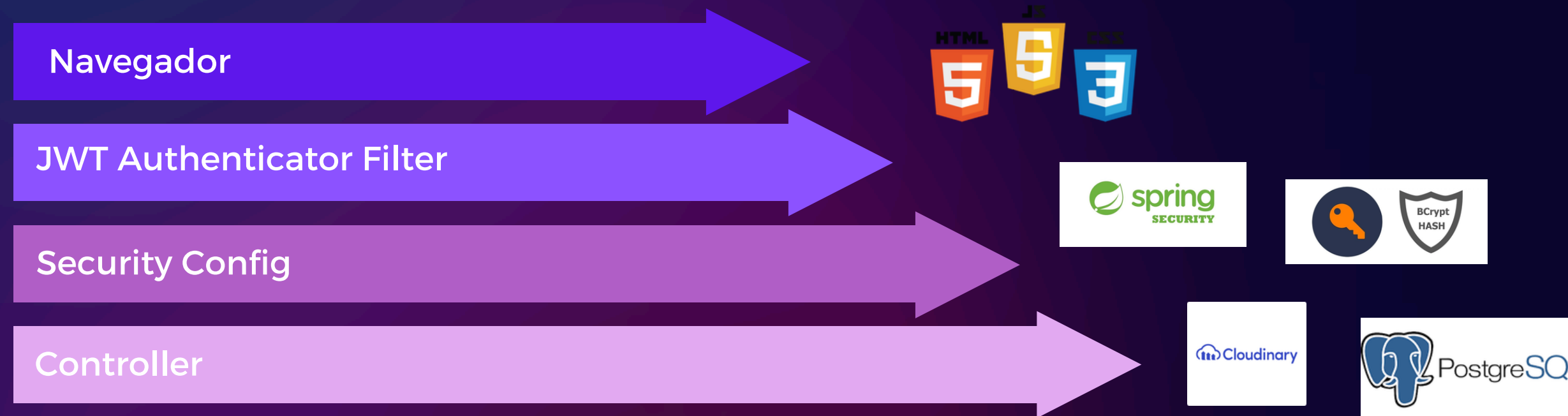
Almacenamiento de Imágenes

Cloudinary como CDN de Imágenes

Frontend

HTML5 + CSS3 + JavaScript vanilla, IntersectionObserver API

Arquitectura del Sistema



Las imágenes se suben directamente a Cloudinary, solo se guarda la URL en la base de datos. El backend así se mantiene ligero y escalable

Stateless con JWT

No hay sesiones en servidor. Cada petición es independiente. El token JWT contiene el email del usuario firmado con HS256 y expira en 24 horas. Esto permite escalar horizontalmente sin estado compartido.

Controller

Valida entrada y devuelve un HTTP response correctamente formateada.

Service

Lógica de negocio y transacciones con @Transactional

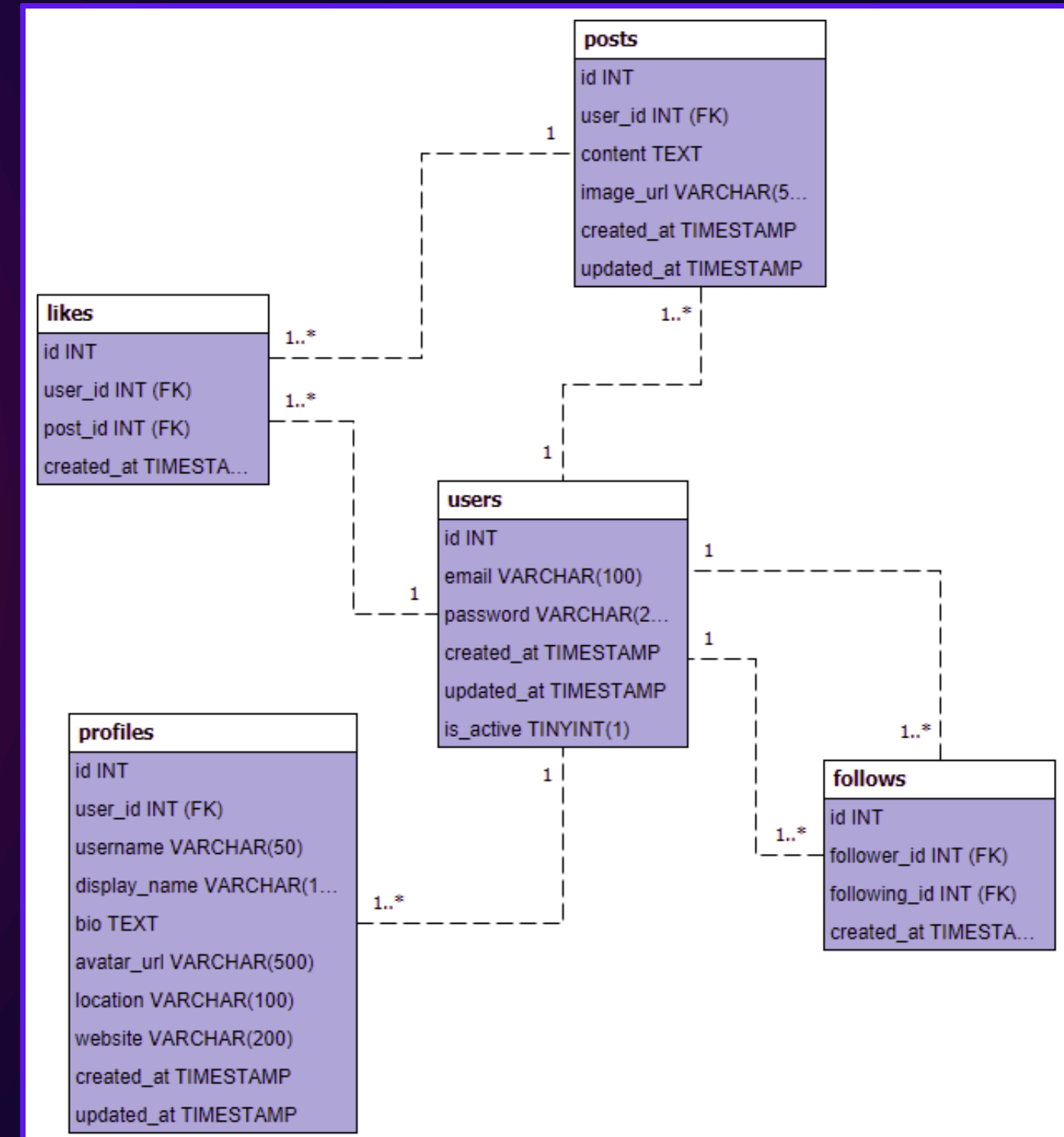
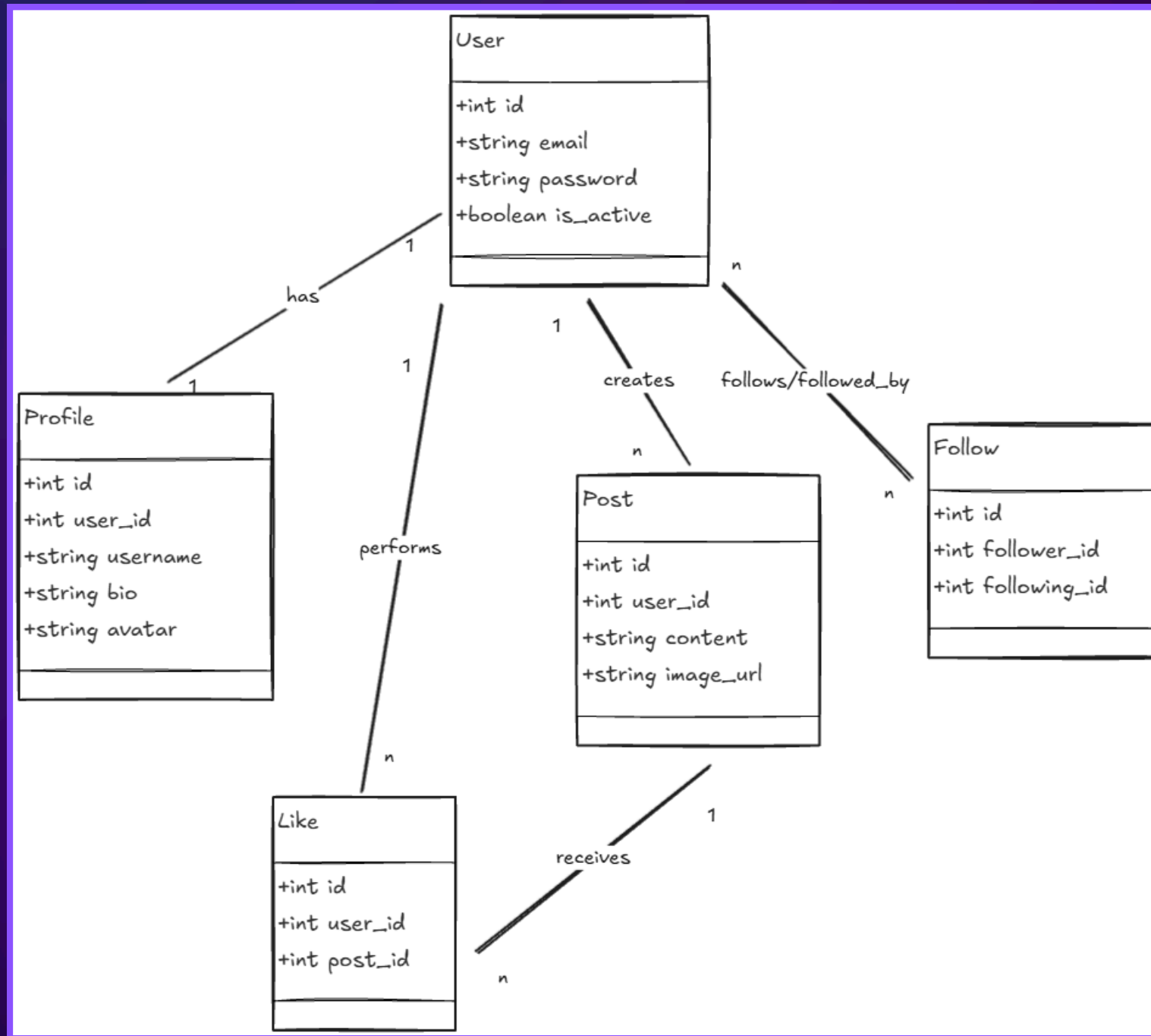
Repository

Acceso a datos con Spring Data JPA

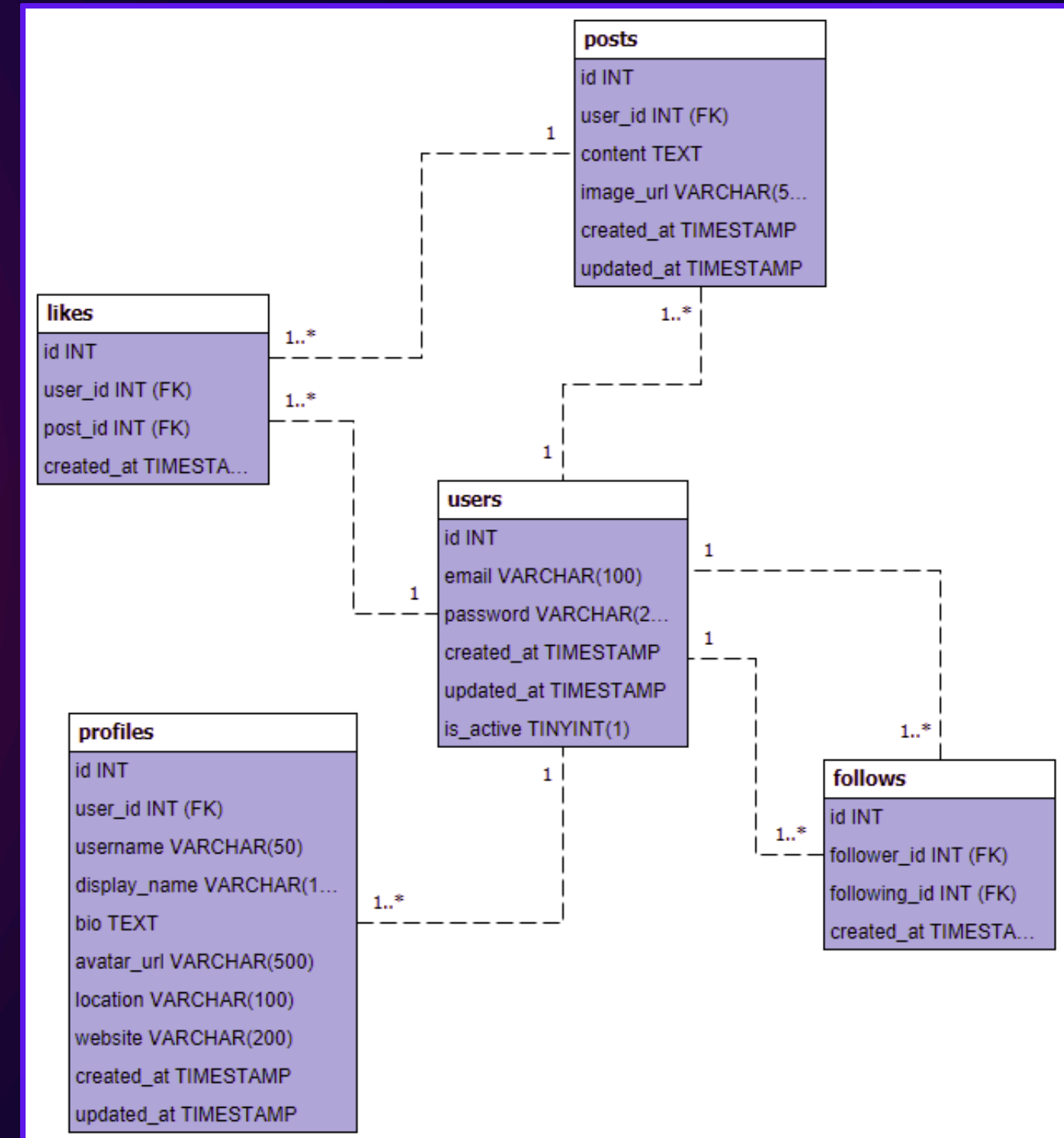
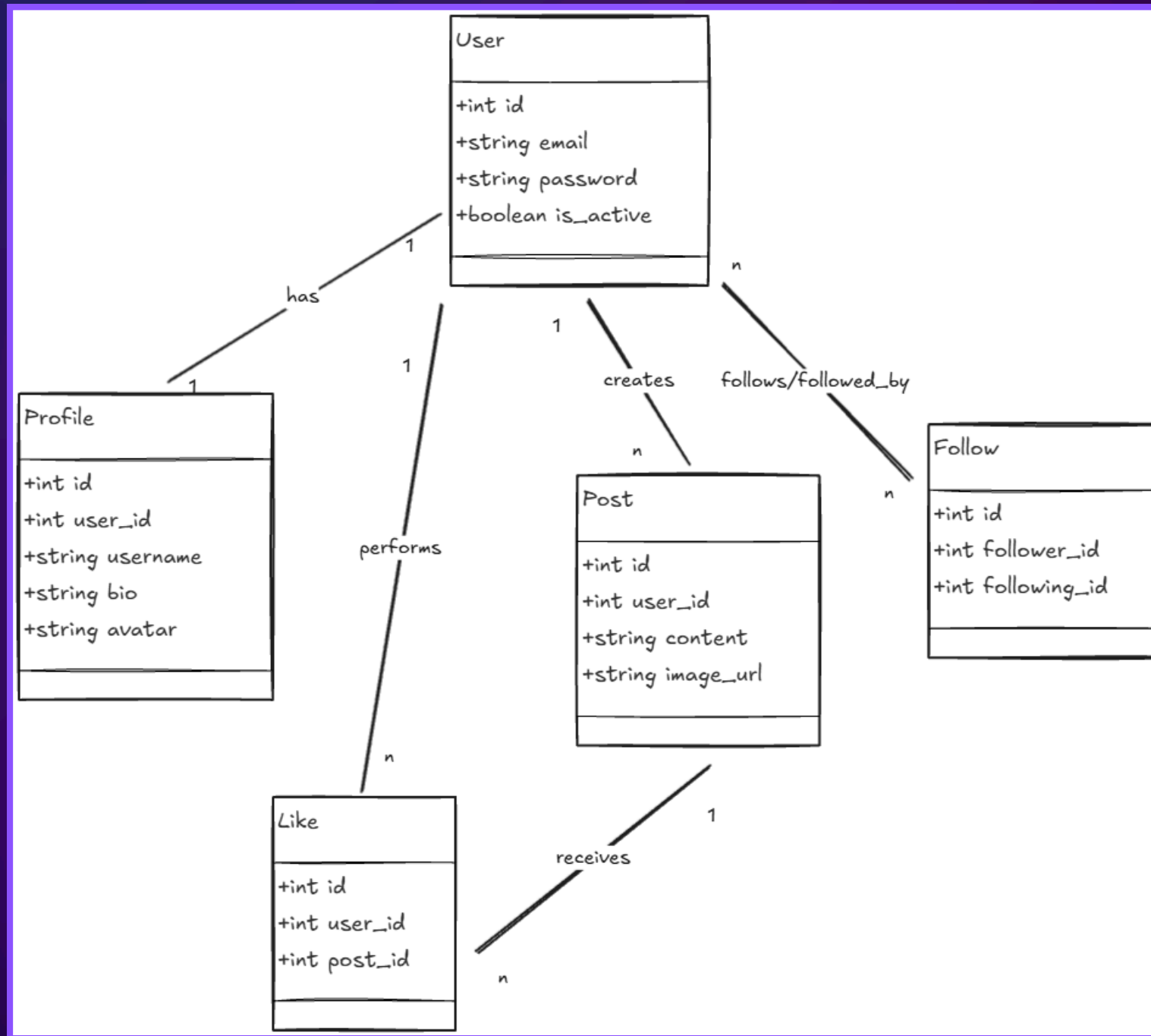
DTO

Nunca se expone la entidad en el cliente

Base de Datos: Esquema y Decisiones



Base de Datos: Esquema y Decisiones



Base de Datos: Esquema y Decisiones



Users

Credenciales y rol (USER / ADMIN), contraseña hashada con BCrypt, campo is_active.



Profiles

Datos públicos: username, bio, avatar_url, ubicación, web. Relación 1:1 con users.



Posts

Publicaciones con texto e imagen opcional.



Follows

Relación N:M entre usuarios (follower_id → following_id).



Likes

Relación N:M entre usuarios y posts.

¿Por qué separar users y profiles?

Users contiene datos sensibles (email, contraseña, rol). **Profiles** contiene datos públicos (username, bio). Esta separación es una decisión de seguridad: los datos de autenticación nunca se mezclan con los datos visibles.

Integridad referencial

Borrar un usuario elimina en cascada sus posts, perfil, follows y likes. La BD nunca queda en estado inconsistente gracias a CascadeType.ALL y orphanRemoval. El campo is_active permite desactivar cuentas sin borrarlas para trazabilidad futura.

Requisitos funcionales y seguridad

Paso 1 - Registro

POST /api/auth/register → BCrypt hashea contraseña → guarda User + Profile → devuelve token JWT

Paso 2 - Login

POST /api/auth/login → AuthenticationManager valida BCrypt → JwtUtil genera token HS256 → frontend guarda en localStorage

Paso 3 - Peticiones autenticadas

Authorization: Bearer {token} → JwtAuthenticationFilter extrae email → busca usuario en BD → registra en SecurityContext

Endpoints / Requisitos funcionales principales

Endpoint	Método	Acceso
GET /api/posts	Feed paginado	Público
POST /api/posts	Crear post con imagen	Autenticado
POST /api/posts/{id}/like	Toggle like	Autenticado
GET /api/profiles/{username}	Ver perfil	Público
PUT /api/profiles/me	Editar perfil	Autenticado
POST /api/follows/{username}	Seguir usuario	Autenticado
POST /api/auth/register y login	Autenticación	Público



Solo los **GET** son públicos. Todas las operaciones de escritura (**POST, DELETE, PUT**) requieren token. Esta distinción evita que usuarios anónimos provoquen ClassCastException en código que asume un usuario autenticado.

Requisitos funcionales y seguridad

Paso 1 - Registro

POST /api/auth/register → BCrypt hashea contraseña → guarda User + Profile → devuelve token JWT

Paso 2 - Login

POST /api/auth/login → AuthenticationManager valida BCrypt → JwtUtil genera token HS256 → frontend guarda en localStorage

Paso 3 - Peticiones autenticadas

Authorization: Bearer {token} → JwtAuthenticationFilter extrae email → busca usuario en BD → registra en SecurityContext

Endpoints / Requisitos funcionales principales

Endpoint	Método	Acceso
GET /api/posts	Feed paginado	Público
POST /api/posts	Crear post con imagen	Autenticado
POST /api/posts/{id}/like	Toggle like	Autenticado
GET /api/profiles/{username}	Ver perfil	Público
PUT /api/profiles/me	Editar perfil	Autenticado
POST /api/follows/{username}	Seguir usuario	Autenticado
POST /api/auth/register y login	Autenticación	Público



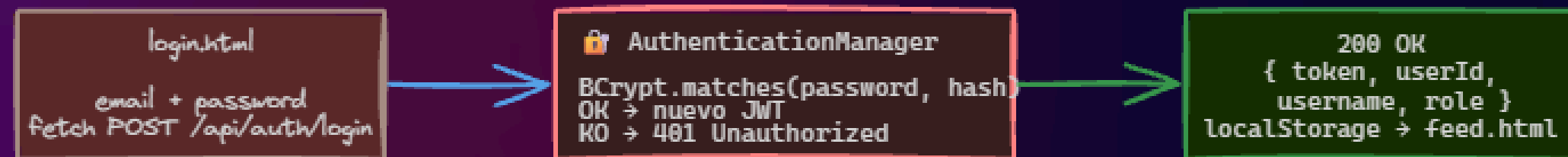
Solo los **GET** son públicos. Todas las operaciones de escritura (**POST, DELETE, PUT**) requieren token. Esta distinción evita que usuarios anónimos provoquen ClassCastException en código que asume un usuario autenticado.

Flujo Frontend - Backend

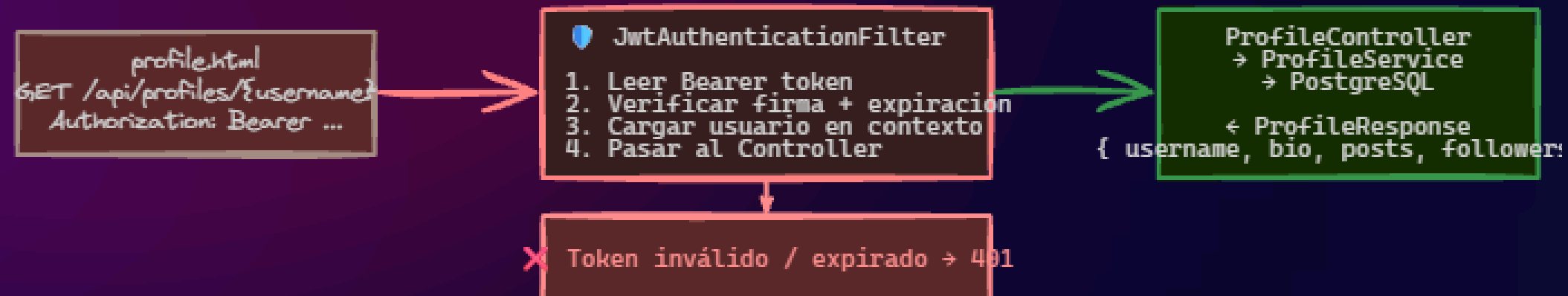
Registro



Login

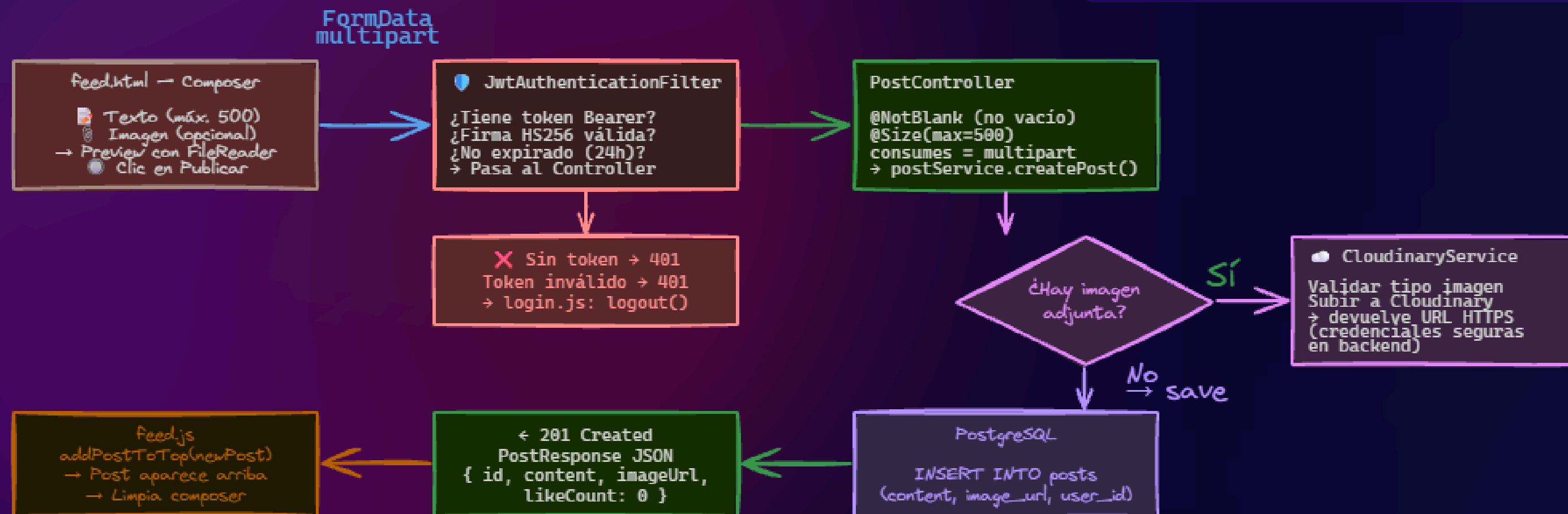


Perfil JWT



Flujo Frontend - Backend

Creación de un Post



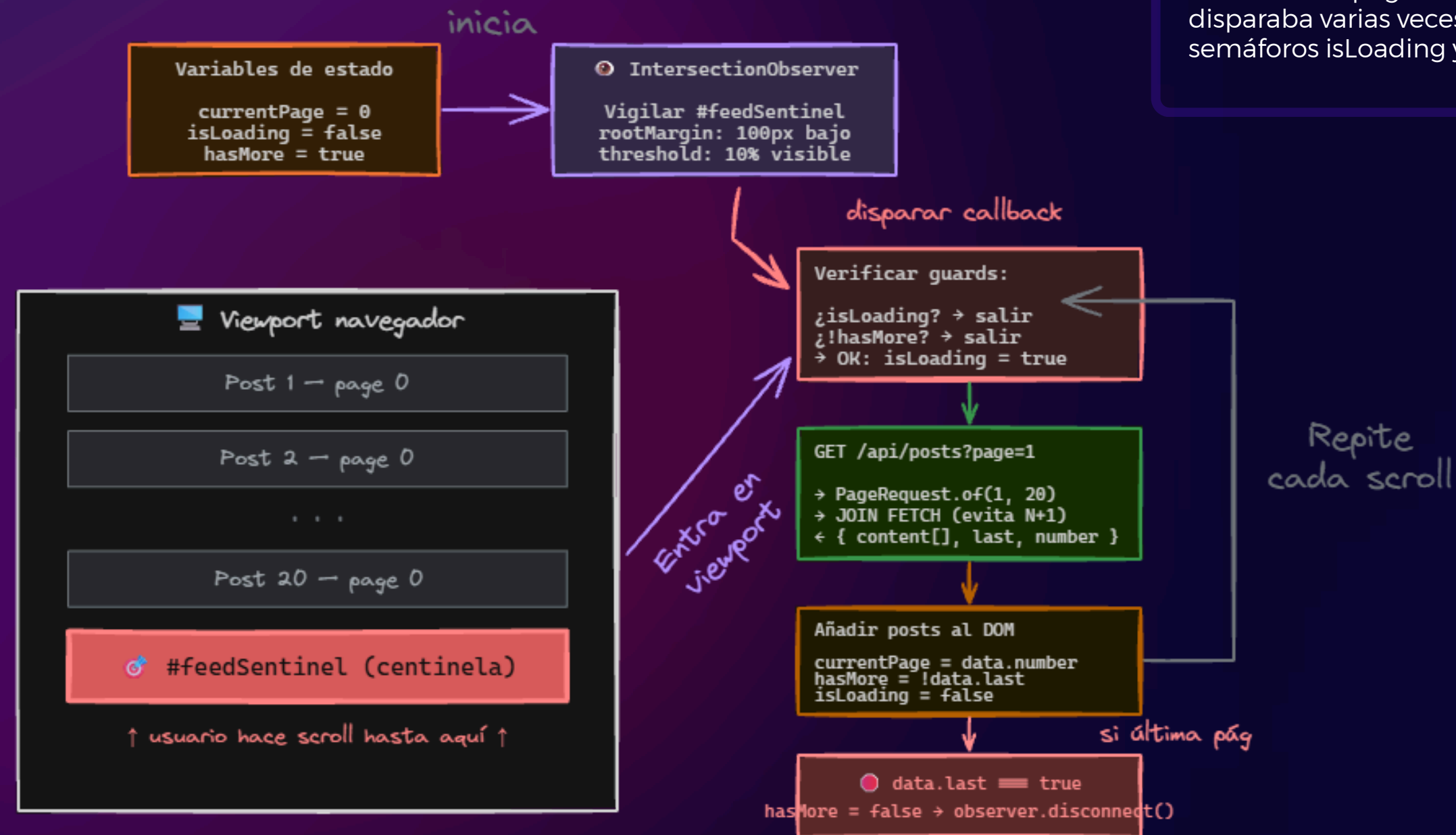
Problema Multipart

JSON no puede transportar binarios. Con FormData, si pones Content-Type manualmente en el fetch, el browser no puede calcular el boundary y el archivo llega corrupto.

Solución: no declarar Content-Type y dejar que lo gestione el browser.

Flujo Frontend - Backend

Scroll Infinito



Problema n+1 y paginación

Problema N+1: sin JOIN FETCH, 20 posts = 21 queries (1 posts + 20 autores). Con JOIN FETCH = 1 sola query. Rendimiento x21.

Problema paginación: el IntersectionObserver se disparaba varias veces seguido. Solución: dos semáforos isLoading y hasMore como porteros.

Frontend: Flujo de la Aplicación

Paso 1 - Login y Registro

Formularios con gradiente azul-morado. Validación en cliente antes de llamar a la API.

Token JWT guardado en localStorage. Redirección automática al feed si ya hay sesión activa.

Paso 2 - Feed

El IntersectionObserver observa un div "sentinel" invisible al final de la lista. Cuando entra en el viewport, carga la siguiente página automáticamente.

Crear posts con texto e imagen opcional (multipart/form-data). Toggle like sin recargar la página. Buscador de usuarios con resultados instantáneos.

Paso 3 - Profile

Formularios con gradiente azul-morado. Validación en cliente antes de llamar a la API.

Token JWT guardado en localStorage. Redirección automática al feed si ya hay sesión activa.

Frontend: Flujo de la Aplicación

Paso 1 - Registro

Crear una cuenta

Únete a nuestra red social

Nombre de usuario

Solo letras, números y guion bajo (3-20 caracteres)

Nombre completo

Email

Contraseña

Seguridad: Débil

Confirmar contraseña

Las contraseñas no coinciden

Crear Cuenta

[¿Ya tienes cuenta? Inicia sesión](#)

Paso 1 - Login

¡Bienvenido de nuevo!

Inicia sesión para continuar

Email

Contraseña

Iniciar Sesión

[¿No tienes cuenta? Regístrate aquí](#)

Paso 2 - Feed

S SOCIAL WEB

F @fabio Salir

F

0 / 500 Publicar

F **Fabio Cuffaro** @fabio hace 15m

¡Acabo de terminar de configurar Spring Security con JWT en mi TFG! La parte de los filtros fue lo más complicado pero ya funciona correctamente.

7 Eliminar

M **María García** @maria_dev hace 2h

Consejo para DAM: lee los stack traces de Java desde abajo hacia arriba. La causa raíz casi siempre está al final. Me costó semanas darme cuenta.

23

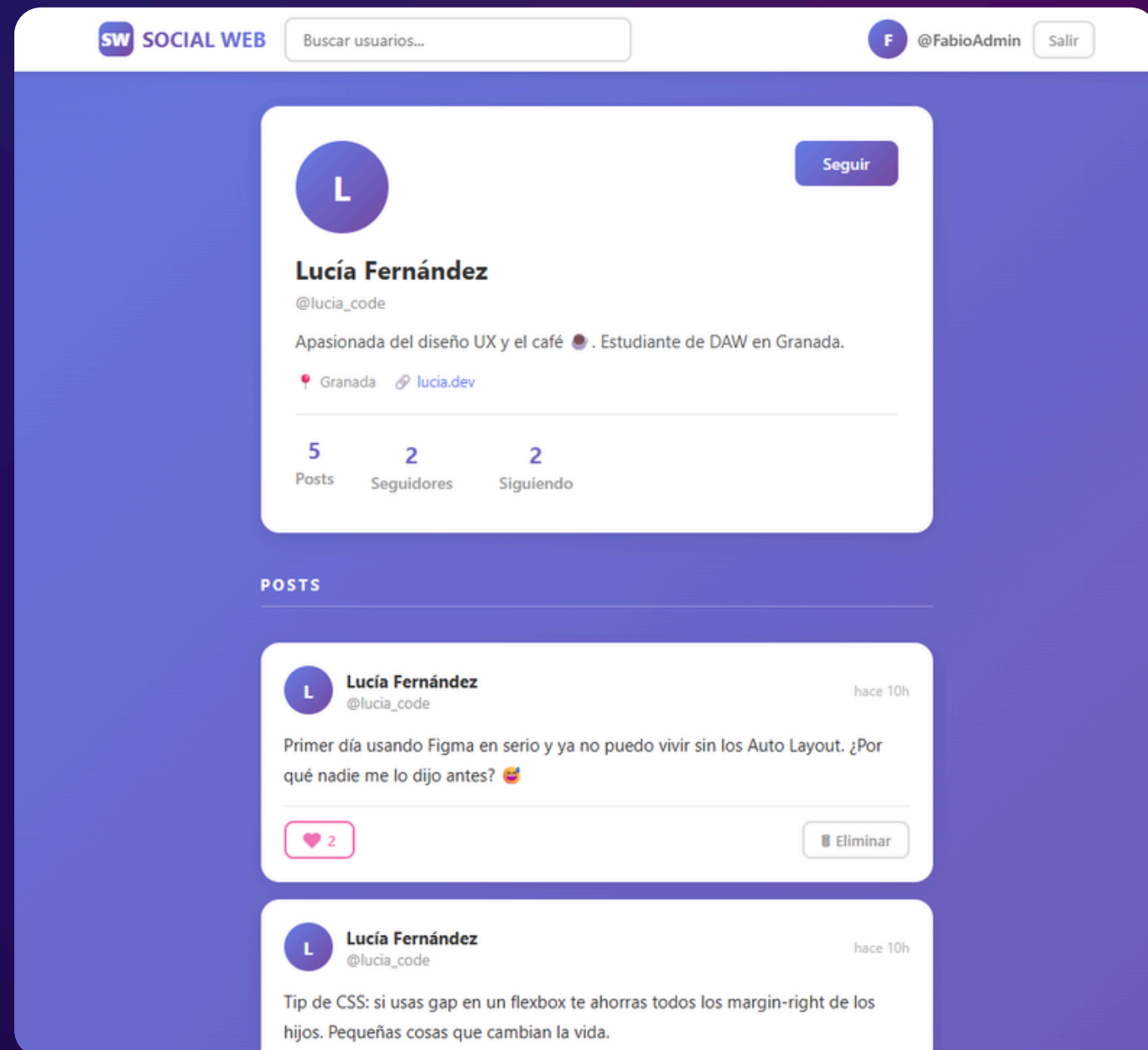
C **Carlos Rodríguez** @carlos_dam hace 5h

PostgreSQL vs MySQL para proyectos de ciclo: me quedé con PostgreSQL. Mejor soporte de tipos y PGAdmin es muy cómodo para gestionar la base de datos.

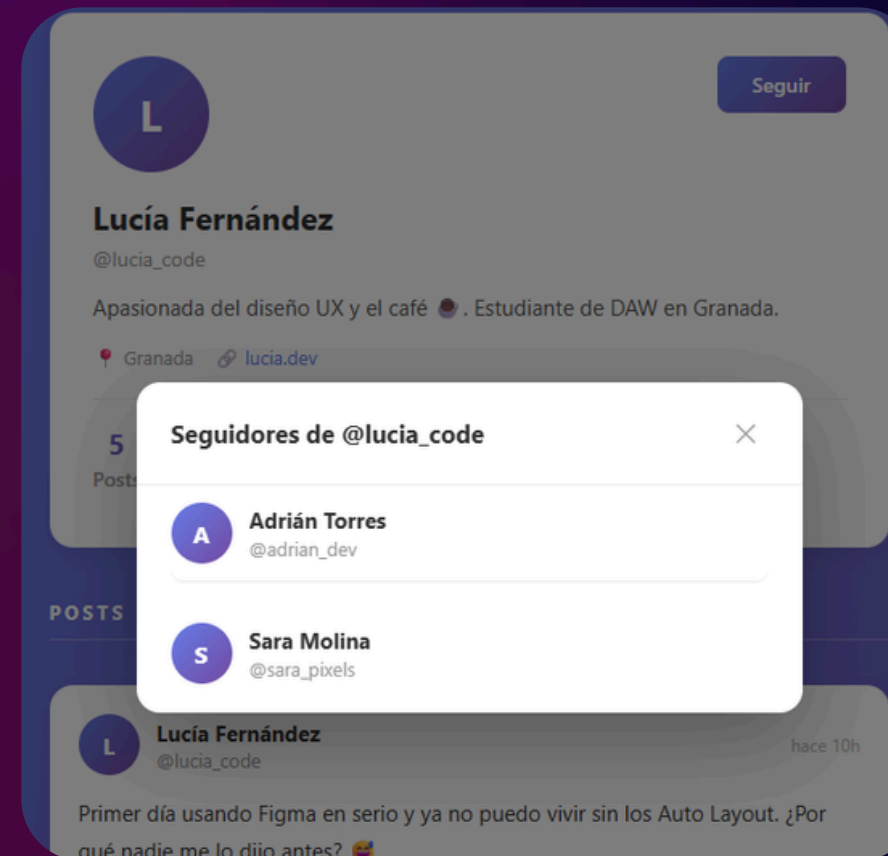
41

Frontend: Flujo de la Aplicación

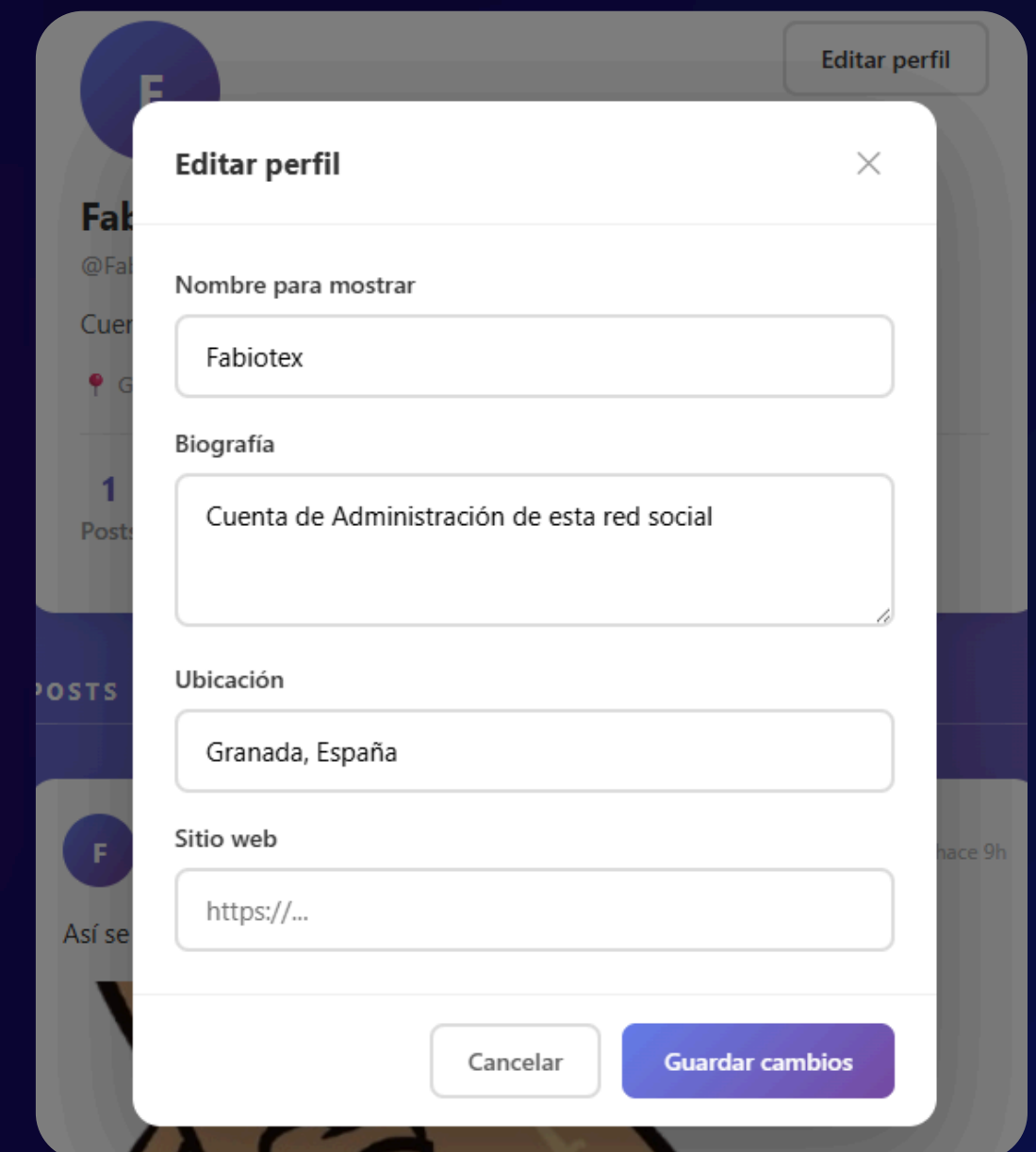
Paso 3 - Profile



Paso 3 - Follows



Paso 3 - Modal Perfil



Fases del Desarrollo del Proyecto

Metodología bottom-up: primero la base de datos, después el backend, después el frontend....

Fase 1

1

Diseño del esquema PostgreSQL

Modelos JPA, repositorios y configuración
Spring Boot

2

Fase 2

Fase 3

3

Autenticación: Registro, login y filtro JWT

API de posts e integración Cloudinary

4

Fase 4

Fase 5

5

Frontend completo: Login, Feed, Profile

Scroll infinito, likes, follows y búsqueda

6

Fase 6

Fase 7

7

Test de integración y corrección de 3 fallos reales



Este orden evita construir interfaces que no coincidan con los datos reales. Cada capa se construye sobre una base sólida y probada.

Así he podido mejorar las versiones en Github y entender mejor el proceso de creación de una app real.

Fases del Desarrollo del Proyecto

Metodología bottom-up: primero la base de datos, después el backend, después el frontend....

Fase 1

1

Diseño del esquema PostgreSQL

Modelos JPA, repositorios y configuración
Spring Boot

2

Fase 2

Fase 3

3

Autenticación: Registro, login y filtro JWT

API de posts e integración Cloudinary

4

Fase 4

Fase 5

5

Frontend completo: Login, Feed, Profile

Scroll infinito, likes, follows y búsqueda

6

Fase 6

Fase 7

7

Test de integración y corrección de 3 fallos reales



Este orden evita construir interfaces que no coincidan con los datos reales. Cada capa se construye sobre una base sólida y probada.

Así he podido mejorar las versiones en Github y entender mejor el proceso de creación de una app real.

Test e Integración: Fallos Reales Corregidos

Test Conectados a PostgreSQL real (sin mocks)

TfgRedSocialApplication Tests

Contexto arranca, BD conectada, repositorios disponibles.

AuthControllerTest

Registro 201, login correcto devuelve token, login incorrecto 401, email inválido 400.

PostControllerTest

Feed 200, crear post 201, post vacío 400, ciclo crear+borrar completo.

SecurityTest

Sin token 401, token falso rechazado, token válido permite acceso.

3 Bugs descubiertos y corregidos

- 1** profileUserId null → "null" enviado al servidor
El IntersectionObserver se disparaba antes de que loadProfile() (async) terminase. Fix: reset de estado al inicio + guard `if (!profileUserId) return`.
- 2** Posts vacíos aceptados (201)
PostController sin validación en @RequestParam content. Fix: @Validated + @NotBlank + handler de ConstraintViolationException → 400.
- 3** Spring Security devolvía 403 en lugar de 401
Sin exceptionHandling configurado, Spring usa Http403ForbiddenEntryPoint. Fix: authenticationEntryPoint explícito (401) + accessDeniedHandler (403).

Despliegue y Trabajos Futuros

Arquitectura de Despliegue

RailWay

PostgreSQL (importar script SQL) + Spring Boot JAR con variables de entorno: BD, JWT, Cloudinary. CI/CD automático con cada push a GitHub.

Vercel

HTML/CSS/JS estático con la carpeta "2 - FRONTEND" como raíz. Cambiar localhost:8080 por la URL pública de Railway antes de desplegar.

Render

Servicios web + PostgreSQL gestionado. Despliegue automático desde GitHub con build & start commands. Variables de entorno (BD, JWT, etc.) y HTTPS incluido. Permite background workers y cron jobs.



Plan gratuito en todas las plataformas. Cada Commit redespiega automáticamente el Frontend y el Backend

Trabajos Futuros



Panel de Administración

Gestión de usuarios con is_active (desactivar sin borrar).



Apache Kafka - Mensajería y WebSockets

Bus de mensajes para chat en tiempo real, arquitectura usada en producción en WhatsApp y LinkedIn.



Notificaciones en la app

Server-Sent Events para likes y nuevos seguidores.



Categorías y Hashtags

Filtrar el feed por temática.



Rediseño del Frontend

Migración a React o Vue.js, dark mode, componentes reutilizables.

Conclusión Final

El aprendizaje más valioso no ha sido solo: entender y comprender las anotaciones(Beans), seguridad, asincronia, fetch... Ha sido entender por qué un bug aparecía. No he parado de lanzar y relanzar el servidor y cada vez que lo hacía había un fallo nuevo, había que mirar la documentación, volver a probar y ver que todo funcionaba..

+100

7

+ 20

+ 8000

Páginas de documentación

Fases de Desarrollo

Fallos, bugs y test corregidos

Lineas de Código

Mi tfg es algo básico, es la base con buena estructura para poder escalar la aplicación en un futuro. No he querido hacerla increíble ni tampoco vistosa con muchos efectos o con un Frontend impactante... Mi principal objetivo ha sido entender la estructura, carpetas, sistemas, flujos... Y me ha servido como base para mis prácticas...

¡Gracias!